

ANÁLISIS TÉCNICO ACADÉMICO

Mantenimiento de Software — Examen FIS EPN

Proyecto: Examen Juan Arroyo

1. Introducción y Marco Conceptual

El presente trabajo responde a un examen práctico de Mantenimiento de Software de la Facultad de Ingeniería en Sistemas (FIS-EPN). Conforme a la taxonomía clásica establecida por las normas ISO/IEC 14764 e IEEE 1219, el mantenimiento se clasifica en cuatro categorías: correctivo, adaptativo, perfectivo y preventivo. El README.md del módulo Python documenta explícitamente esa correspondencia mediante referencias a artefactos concretos del código fuente.

El núcleo funcional exigido es un CRUD (Create, Read, Update, Delete) sobre la entidad gasto, implementado como API REST y consumido desde una interfaz web mínima. El alcance del mantenimiento supera la mera corrección de fallos: comprende también el endurecimiento del sistema, la adaptación a políticas institucionales (clave FIS), la mejora de la calidad mediante pruebas y documentación, y la prevención de fallos futuros a través de validación exhaustiva, registro de eventos y manejo controlado de errores.

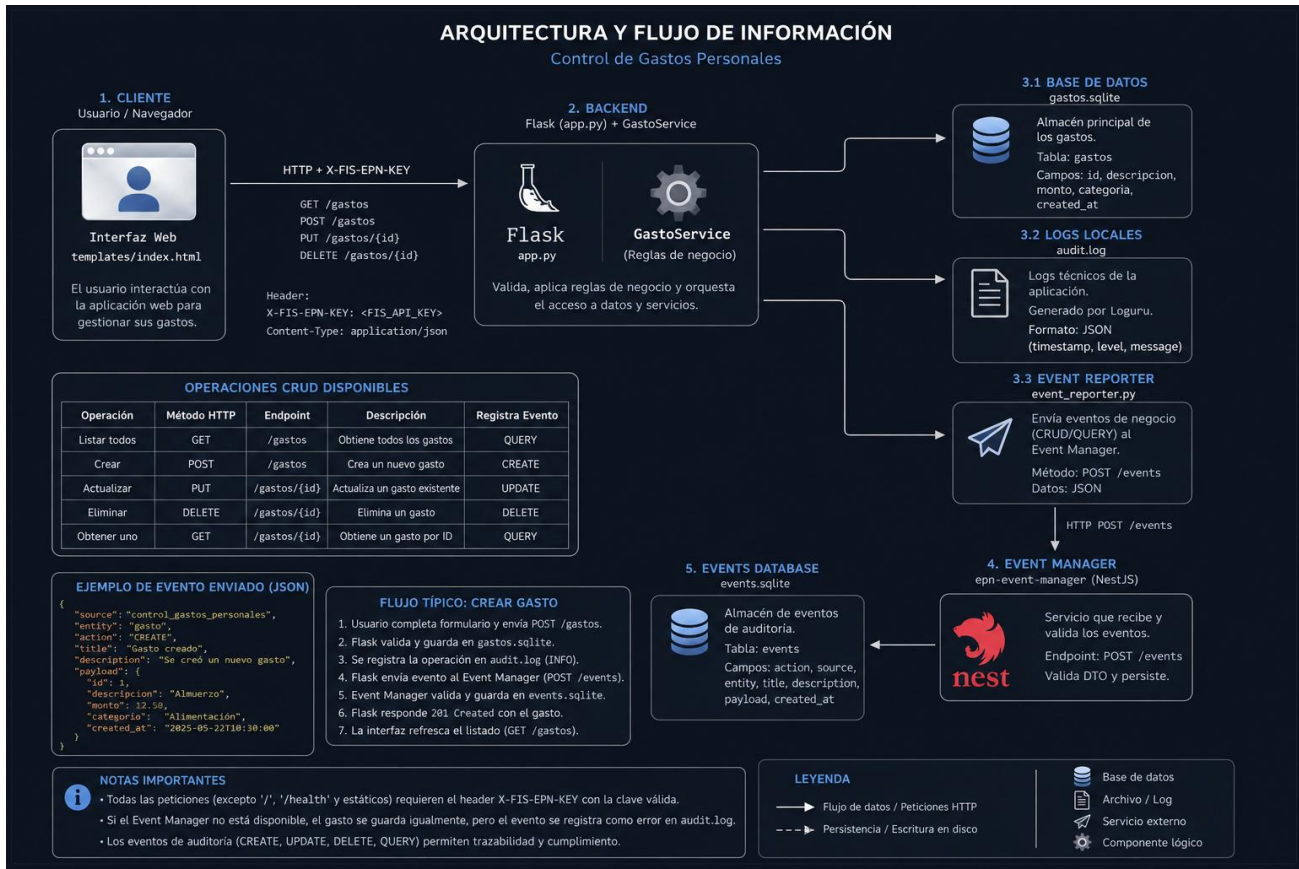
2. Arquitectura del Sistema

2.1 Visión General

La solución adopta una arquitectura bimonolítica distribuida en dos servicios cooperantes:

Componente	Tecnología	Rol
<i>control_gastos_personales</i>	Python 3, Flask, SQLite	CRUD de gastos, validación, autenticación, registros locales
<i>epn-event-manager</i>	NestJS, TypeORM, SQLite	Registro centralizado de eventos de dominio (auditoría)

A continuación se muestra el flujo de comunicación entre ambos componentes:



2.2 Capas del Módulo Python

El módulo Python aplica un diseño en capas que facilita el mantenimiento independiente de cada componente y la redacción de pruebas focalizadas:

Presentación	app.py · templates/index.html
Aplicación	gasto_service.py
Dominio / Validación	validators.py
Persistencia	gasto_repository.py · database.py
Infraestructura transversal	config.py · logging_config.py · event_reporter.py

Este desacoplamiento constituye, en sí mismo, una decisión de mantenimiento perfecto: las operaciones CRUD no están acopladas directamente a sentencias SQL dentro de los controladores.

3. Implementación del CRUD

3.1 Modelo de Datos

La tabla gastos reside en una base de datos SQLite (db/gastos.sqlite) y contiene los siguientes campos:

- id — INTEGER, clave primaria con autoincremento opcional.
- descripcion — TEXT, obligatorio.
- monto — REAL, obligatorio.
- categoria — TEXT, valor por defecto 'Otros'.
- created_at — TEXT, marca de tiempo generada automáticamente.

3.2 Operaciones REST

Operación	Método y ruta	Código éxito	Comportamiento principal
Crear	POST /gastos	201	Valida el cuerpo, admite id explícito, rechaza duplicados (409)
Listar	GET /gastos	200	Orden ascendente por id
Obtener uno	GET /gastos/{id}	200 / 404	Recurso individual
Actualizar	PUT /gastos/{id}	200 / 404	Actualización parcial; valida JSON nulo
Eliminar	DELETE /gastos/{id}	200 / 404	Elimina y devuelve confirmación

Las rutas auxiliares incluyen GET / para la interfaz de usuario y GET /health para la verificación del estado del servicio (sin autenticación requerida).

3.3 Reglas de Negocio

Las invariantes del dominio se definen en validators.py y son aplicadas por GastoService antes de cualquier operación de persistencia:

- Descripción: cadena no vacía, máximo 200 caracteres, con eliminación de espacios extremos (strip()).
- Monto: valor numérico estrictamente mayor que cero, redondeado a dos decimales.
- Categoría: valor perteneciente al conjunto cerrado {Comida, Transporte, Educación, Salud, Otros}.
- Cuerpo JSON obligatorio; en operaciones PUT parciales, se rechaza la solicitud si no contiene campos válidos.
- Control de IDs duplicados tanto en la creación como en el cambio de identificador (HTTP 409).

La suite de pruebas en tests/test_gasto_service.py certifica la persistencia en SQLite, el rechazo de montos negativos e IDs duplicados, la actualización con cuerpo JSON nulo y el borrado de registros inexistentes (6 casos documentados).

4. Tipos de Mantenimiento Realizados

4.1 Mantenimiento Correctivo

El mantenimiento correctivo aborda la corrección de defectos y comportamientos incorrectos ante entradas inválidas o estados inconsistentes del sistema:

<i>Problema abordado</i>	<i>Evidencia en el código</i>
<i>PUT con cuerpo JSON nulo o inválido</i>	<code>request.get_json(silent=True) + validate_gasto_payload(..., partial=True); prueba</code> <code>test_update_sin_json_no_rompe_servicio</code>
<i>IDs duplicados al crear o actualizar Recursos inexistentes</i>	<code>GastoService.create/update</code> con <code>exists()</code> y HTTP 409 HTTP 404 en GET, PUT y DELETE
<i>Fallos del Event Manager</i>	EventReporter registra WARNING/ERROR sin interrumpir el CRUD
<i>Errores no controlados en los endpoints</i>	Bloques try/except por ruta con respuesta 500 genérica y registro ERROR

El mantenimiento correctivo en este proyecto no se limita a capturar excepciones: normaliza también los códigos de estado HTTP y los mensajes de error en formato JSON `{'error': '...'}`, lo que mejora la predictabilidad de la API.

4.2 Mantenimiento Adaptativo

El mantenimiento adaptativo recoge los ajustes necesarios para que el sistema opere en el entorno institucional EPN/FIS:

<i>Requisito adaptativo</i>	<i>Implementación</i>
<i>Autenticación institucional</i>	Cabecera obligatoria X-FIS-EPN-KEY verificada en <code>@app.before_request (app.py)</code>
<i>Configuración externalizada</i>	<code>config.py + .env_config</code> / variables de entorno (FIS_API_KEY, EVENT_MANAGER_URL, etc.)
<i>Integración con sistema de eventos</i>	Cliente HTTP EventReporter hacia <code>epn-event-manager</code>
<i>Interfaz adaptada al esquema de seguridad</i>	<code>index.html</code> inyecta la clave y la incluye en cada llamada fetch

El Event Manager (NestJS) actúa como subsistema adaptativo complementario: expone un contrato REST estable para que cualquier aplicación FIS registre eventos sin acoplar la lógica de auditoría al CRUD de gastos.

4.3 Mantenimiento Perfectivo

El mantenimiento perfectivo agrupa las mejoras de legibilidad, testabilidad y documentación que no alteran los requisitos funcionales externos:

<i>Mejora</i>	<i>Artefacto</i>
<i>Pruebas unitarias automatizadas</i>	<code>pytest</code> en <code>tests/test_gasto_service.py</code> (6 casos)
<i>Documentación del contrato API</i>	<code>docs/openapi.yaml</code> (OpenAPI 3.0.3)

README operativo	Instrucciones de despliegue, variables de entorno y ejemplos curl
Arquitectura modular	Repository + Service + Validators
Segundo servicio con buenas prácticas	NestJS con DTOs (class-validator), migraciones TypeORM y módulos events, stats, health, dashboard
Calidad en TypeScript	ESLint, Prettier y pruebas e2e de base

4.4 Mantenimiento Preventivo

Las acciones de mantenimiento preventivo buscan reducir la probabilidad de fallos futuros y facilitar el diagnóstico ante incidencias:

Medida preventiva	Detalle técnico
Registros de auditoría estructurados	logging_config.py con Loguru; formato JSON (timestamp, level, message); salida a consola y archivo logs/audit.log con rotación diaria
Sanitización de entrada	Límites de longitud, verificación de tipos y enumeración de categorías permitidas
Manejo defensivo por endpoint	Bloques try/except en cada handler de app.py
Health checks	GET /health en Flask; GET /health con verificación de base de datos en NestJS
Validación global en Event Manager	ValidationPipe con whitelist y forbidNonWhitelisted en main.ts
Timeouts en integración	requests.post(..., timeout=5) en EventReporter

Ejemplo de entrada registrada en logs/audit.log:

```
{"timestamp":"2026-05-28T17:08:25.966-05:00","level":"INFO","message":"Gasto actualizado id=1"}
```

5. Elementos Agregados al Sistema (Inventario Técnico)

5.1 Módulo control_gastos_personales

Categoría	Componentes
Observabilidad	logging_config.py · directorio logs/ · audit.log
Seguridad	Middleware de API key; exclusión de / y /health
Persistencia	database.py · gasto_repository.py · db/gastos.sqlite
Calidad de datos	validators.py · excepción ValidationError con status_code
Integración	event_reporter.py (mapeo READ_* → QUERY)
Configuración	config.py · .env.example · .env_config
Interfaz de usuario	templates/index.html (Bootstrap 5, CRUD vía Fetch API)
Documentación	README.md · docs/openapi.yaml
Pruebas	tests/test_gasto_service.py
Dependencias	Flask, requests, python-dotenv, loguru, pytest (requirements.txt)

5.2 Módulo epn-event-manager

Módulo	Función
events	POST /events; consultas por id, fuente y entidad
stats	Contadores agregados por tipo de acción
health	Verificación de conexión SQLite
dashboard	Panel estático en /dashboard
Base de datos	Entidad EventEntity · migración CreateEventsTable · events.sqlite

Cada operación CRUD sobre gastos dispara un evento de dominio con metadatos completos (source, entity, action, title, description, payload), alineado con el enum CREATE | UPDATE | DELETE | QUERY definido en el DTO de NestJS.

6. Integración entre Sistemas

El siguiente diagrama describe, paso a paso, el flujo completo al crear un gasto:

1	Cliente envía POST /gastos con X-FIS-EPN-KEY y JSON válido
2	GastoService valida el cuerpo y persiste mediante GastoRepository
3	app.py invoca event_reporter.report('CREATE', ...)
4	EventReporter normaliza la acción y envía POST a EVENT_MANAGER_URL
5a	[Éxito] Entrada INFO en audit.log — confirmación local y remota
5b	[Fallo] Gasto ya persistido — WARNING/ERROR en audit.log; trazabilidad centralizada degradada

Esta integración materializa el patrón de auditoría desacoplada: la disponibilidad del CRUD tiene prioridad sobre la consistencia fuerte con el sistema de auditoría. Si el Event Manager no responde, el gasto permanece persistido y el fallo queda registrado localmente como WARNING o ERROR.

7. Evaluación de Atributos de Calidad

Atributo	Evaluación
Mantenibilidad	Alta en Python gracias a la separación por capas y a las pruebas focalizadas en el servicio; el Event Manager sigue una organización modular por dominio en NestJS.
Seguridad	API key en cabecera HTTP (adecuada para entorno de examen/desarrollo; no reemplaza OAuth en producción).
Confiabilidad	Manejo de errores por capa con health endpoints activos en ambos servicios.
Observabilidad	Registros JSON locales más almacén de eventos centralizado.

Verificabilidad	Suite pytest documentada; configuración de lint y pruebas en NestJS.
Portabilidad	SQLite embebido; configuración completa mediante variables de entorno.

8. Correspondencia con el Enunciado del Examen

Expectativa del examen FIS	Cumplimiento en el repositorio
CRUD funcional en Python	Completo en Flask con persistencia SQLite.
Clasificación de actividades de mantenimiento	Documentada y trazable a artefactos concretos del repositorio.
Registro y trazabilidad (logs)	logs/audit.log con Loguru en formato JSON.
Integración con sistema externo	epn-event-manager (NestJS).
Mejora de calidad	Pruebas automatizadas, especificación OpenAPI y validación estricta de datos.
Adaptación a la política EPN	Cabecera X-FIS-EPN-KEY obligatoria en todas las rutas protegidas.

9. Conclusiones

El proyecto entregado en Examen_JuanArroyo constituye una implementación completa de un CRUD de gastos personales en Python, enriquecida con actividades explícitas de mantenimiento en las cuatro categorías de la ingeniería de software:

- Correctivo: corrección de casos límite (JSON nulo, IDs duplicados, códigos HTTP coherentes 404/409/500).
- Adaptativo: autenticación FIS mediante cabecera, variables de entorno e integración HTTP con el Event Manager.
- Perfectivo: arquitectura en capas, pruebas automatizadas, documentación OpenAPI y subsistema NestJS estructurado.
- Preventivo: validación estricta, registro estructurado con rotación diaria, sanitización de entrada y health checks.

Los registros de auditoría (logging_config.py + logs/audit.log) representan el componente de observabilidad más visible, pero el alcance del mantenimiento es sustancialmente más amplio: abarca validadores, el reporter de eventos, la documentación de la API, las pruebas unitarias, la configuración externalizada, la interfaz web y un segundo servicio dedicado a la trazabilidad de operaciones CRUD.

Desde una perspectiva académica, el trabajo demuestra que el mantenimiento de software no es un apéndice posterior al desarrollo, sino un conjunto coordinado de prácticas que recorre todo el ciclo de vida del CRUD: desde la entrada del usuario hasta la persistencia y la auditoría institucional.

10. Referencias Técnicas del Repositorio

Elemento	Referencia
Commit de entrega	1db619d — Examen (28 de mayo de 2026)
Módulo principal CRUD	control_gastos_personales/
Sistema de eventos	ept-event-manager/
Documentación operativa (Python)	control_gastos_personales/README.md
Documentación operativa (NestJS)	ept-event-manager/README.md
Repositorio	ARROYO-BAQUERIZO-JUAN-CARLOS-constswgr2